



2º CUATRIMESTRE

ACTIVIDAD 8: PROGRAMACIÓN ORIENTADA A OBJETOS (POO) EN C++ — JUEGO DE NAVES EN CONSOLA (class Nave) - LPR 5º 3º A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

1. OBJETIVOS Y TRANSICIÓN: DE STRUCT A CLASS

Esta actividad está diseñada para aprender haciendo. En la Actividad 7 repasamos un struct para agrupar variables (ej. id, nombre, resultado).

La **Programación Orientada a Objetos (POO)** es simplemente el siguiente paso evolutivo. Una **Clase (class)** es exactamente igual a un struct, pero con dos superpoderes:

1. **Privacidad (private):** Podemos ocultar variables para que nadie las modifique por accidente desde afuera.
2. **Acciones (Métodos):** Podemos meter funciones *adentro* de la clase para que el objeto haga cosas por sí mismo (ej. `miNave.disparar()`).

¡Recuerden!, utilizamos struct para agrupar variables (ej. id, nombre, cuota). La POO no es una magia nueva; es la evolución natural de ese concepto.

3. En **Lenguaje Estructurado (struct)**: Agrupás los datos, pero las funciones que modifican esos datos están sueltas por todo el código.
4. En **POO (class)**: Agrupás los datos (ahora llamados **Atributos**) Y las funciones que los modifican (ahora llamadas **Métodos**) dentro de una misma "caja fuerte" llamada **Objeto**.

La regla de oro de la seguridad (Encapsulamiento):

En POO, las variables internas de un objeto se declaran como **private** (privadas). Nadie desde afuera puede modificarlas directamente con un `=`. Para interactuar con la clase, usamos métodos **public** (públicos) que actúan como "puertas controladas".

En esta actividad desarrollarán un **Simulador de Combate Espacial en Consola**, definiendo la clase Nave.

Objetivos Clave:

1. **Diseño de Clases (class):** Definir atributos privados (private:) y métodos públicos (public:).
2. **Encapsulamiento:** Proteger la salud y energía de la nave para que no acepten valores negativos.
3. **Documentación Técnica:** Redactar el Informe APA v7, el Manual del Programador y el Manual del Usuario basándose estrictamente en las plantillas provistas en la actividad.

2. METODOLOGÍA DE APRENDIZAJE 5R (No es solo "copiar y pegar")

El código fuente se provee para que no se traben con errores de sintaxis, pero **deben analizarlo**:

1. **R1 - REGISTRAR:** Copia el código fuente en src/main.cpp, compílalo y verifica su ejecución. (Controles: W, A, S, D para mover, ESPACIO para disparar).
2. **R2 - REDUCIR:** Identifica dónde termina la sección private: (datos) y dónde empieza la public: (acciones).
3. **R3 - RESPONDER:** Responde el Cuestionario Técnico en tu Informe APA v7, demostrando que entendiste *por qué* el código funciona.
4. **R4 - REFLEXIONAR:** Implementa UNA de las 15 funcionalidades extra para aprobar.
5. **R5 - REVISAR:** Sube todo al repositorio de GitHub y GitLab respetando la estructura de carpetas.

3. ESTRUCTURA OBLIGATORIA DEL REPOSITORIO EN GITHUB

Ojo, repito no se aceptan archivos sueltos. Su espacio de trabajo local debe estar organizado así:

juegodenaves/

```
|— .gitignore          <-- Exclusiones (.exe, *.out, .vscode/)
|— LICENSE             <-- Licencia MIT escolar
|— README.md          <-- Portada del proyecto
|— docs/              <-- Carpeta de Documentación Formal
|   |— InformeEEST1_LPR2026_ACT08_G03_v1.0.0.pdf <-- Informe en formato APA v7
|   |— CHANGELOG.md    <-- Historial de versiones
|   |— manuales/
|       |— manual_programador_v1.0.0.pdf <-- Manual Técnico (Ver plantilla)
|       |— manual_usuario_v1.0.0.pdf   <-- Guía de uso (Ver plantilla)
|— src/                <-- Código fuente C++
|   |— main.cpp
|— capturas/           <-- Evidencias de ejecución (.png)
|   |— combate_consola.png <-- Captura de tu consola con TU nombre y apellido
```

4. CÓDIGO FUENTE (src/main.cpp)

Este código está explicado línea por línea. Lean mientras lo transcriben.

```
/*
=====
Materia: Laboratorio de Programación (LPR) — 5º Año
Institución: E.E.S.T. N° 999 "Österreichische Schule der Nationalökonomie" — Vicente López
Archivo: src/main.cpp
Actividad 8: Introducción a POO - Motor de Juego de Naves (Clases, Objetos y Constructores)
Docente: Mansilla Muñoz York
Estudiante: Murray Rothbard
=====
*/

#include <iostream>
#include <windows.h>
#include <conio.h>
#include <vector>
#include <ctime>
#include <cctype> // Para tolower()

using namespace std;

void gotoxy(int x, int y) {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    COORD dwPos;
    dwPos.X = x; dwPos.Y = y;
    SetConsoleCursorPosition(hCon, dwPos);
}

void ocultarCursor() {
    HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cci;
    cci.dwSize = 2; cci.bVisible = FALSE;
    SetConsoleCursorInfo(hCon, &cci);
}

// =====
// CLASE NAVE (El Jugador)
// =====
class Nave {
private:
    int x, y;
    int vidas;

public:
    Nave(int _x, int _y, int _vidas) {
        x = _x; y = _y; vidas = _vidas;
    }

    int getX() { return x; }
    int getY() { return y; }
    int getVidas() { return vidas; }
    void reducirVida() { vidas--; }

    void pintar() {
        gotoxy(x, y); cout << " ^ ";
        gotoxy(x, y + 1); cout << " (O) ";
        gotoxy(x, y + 2); cout << "^^^^ ";
    }

    void borrar() {
        gotoxy(x, y); cout << " ";
        gotoxy(x, y + 1); cout << " ";
        gotoxy(x, y + 2); cout << " ";
    }

    void mover(char tecla) {
        borrar();
        if (tecla == 'a' && x > 2) x--;
        if (tecla == 'd' && x < 70) x++;
        if (tecla == 'w' && y > 3) y--;
        if (tecla == 's' && y < 20) y++;
        pintar();
    }
}
```

```
}
};

// =====
// CLASE ASTEROIDE Y BALA (Simplificadas)
// =====
class Asteroide {
private:
    int x, y;
public:
    Asteroide(int _x, int _y) { x = _x; y = _y; }
    int getX() { return x; }
    int getY() { return y; }
    void pintar() { gotoxy(x, y); cout << "O"; }
    void borrar() { gotoxy(x, y); cout << " "; }
    void mover() {
        borrar();
        y++;
        if (y > 23) { y = 4; x = (rand() % 70) + 2; }
        pintar();
    }
};

class Bala {
private:
    int x, y;
public:
    Bala(int _x, int _y) { x = _x; y = _y; }
    int getX() { return x; }
    int getY() { return y; }
    void mover() {
        gotoxy(x, y); cout << " ";
        y--;
        gotoxy(x, y); cout << "|";
    }
    bool fueraDeRango() { return y < 4; }
};

// =====
// PROGRAMA PRINCIPAL
// =====
int main() {
    srand(time(NULL));

    // SISTEMA ANTI-COPIA: Petición de datos
    string nombrePiloto;
    cout << "Ingresa tu Nombre y Apellido (Se mostrara en la captura de pantalla): ";
    getline(cin, nombrePiloto);

    system("cls");
    ocultarCursor();
    bool gameOver = false;

    Nave jugador(35, 20, 3);
    vector<Asteroide*> asteroides;
    vector<Bala*> balas;

    for (int i = 0; i < 5; i++) {
        asteroides.push_back(new Asteroide((rand() % 70) + 2, (rand() % 5) + 4));
    }

    jugador.pintar();

    while (!gameOver) {
        // HUD CON NOMBRE DEL ESTUDIANTE
        gotoxy(2, 1); cout << "PILOTO: " << nombrePiloto << " ";
        gotoxy(2, 2); cout << "VIDAS: " << jugador.getVidas() << " ";

        if (_kbhit()) {
            char tecla = tolower(_getch()); // tolower evita el problema del Bloq Mayús
            if (tecla == ' ') {
                balas.push_back(new Bala(jugador.getX() + 2, jugador.getY() - 1));
            } else {
                jugador.mover(tecla);
            }
        }
    }
}
```

```
for (int i = 0; i < balas.size(); i++) {
    balas[i]->mover();
    if (balas[i]->fueraDeRango()) {
        gotoxy(balas[i]->getX(), balas[i]->getY()); cout << " ";
        delete balas[i];
        balas.erase(balas.begin() + i);
    }
}

for (int i = 0; i < asteroides.size(); i++) {
    asteroides[i]->mover();

    // Colisión Asteroide -> Jugador
    if ((asteroides[i]->getX() >= jugador.getX() && asteroides[i]->getX() <= jugador.getX() + 4) &&
        (asteroides[i]->getY() >= jugador.getY() && asteroides[i]->getY() <= jugador.getY() + 2)) {
        jugador.borrar();
        jugador.reducirVida();
        asteroides[i]->borrar();
        asteroides[i] = new Asteroide((rand() % 70) + 2, 4);
        jugador.pintar();

        if (jugador.getVidas() == 0) gameOver = true;
    }

    // Colisión Bala -> Asteroide
    for (int j = 0; j < balas.size(); j++) {
        if (asteroides[i]->getX() == balas[j]->getX() && (asteroides[i]->getY() == balas[j]->getY() || asteroides[i]->getY() + 1 == balas[j]->getY())) {
            gotoxy(balas[j]->getX(), balas[j]->getY()); cout << " ";
            delete balas[j];
            balas.erase(balas.begin() + j);

            asteroides[i]->borrar();
            asteroides[i] = new Asteroide((rand() % 70) + 2, 4);
            break;
        }
    }
}
Sleep(30);
}

system("cls");
gotoxy(30, 10);
cout << "G A M E   O V E R... O sea... Perdiste!!!";
gotoxy(30, 12);
system("pause");
return 0;
}
```

5. DESAFÍO OBLIGATORIO: IMPLEMENTACIÓN DE FUNCIONALIDAD ÚNICA

El código base mostrado es solo el motor. Para aprobar, debés elegir UNA de las [15 funcionalidades](#) publicadas en el anexo de Classroom/Moodle, analizar su código e integrarla correctamente en el **main.cpp**.

6. CUESTIONARIO TÉCNICO DE CONTROL (Para el Informe APA)

Responde esto en la sección "Desarrollo" de tu informe APA v7:

1. **struct vs class:** ¿Cuál es la principal diferencia técnica entre un struct (visto en Act. 7) y una clase, especialmente respecto a la sección private y public?
2. **Encapsulamiento:** ¿Por qué es peligroso dejar la variable salud como pública? ¿Qué desastre podría causar un programador novato desde el main()?
3. **Constructor:** Explicá con tus palabras para qué sirve el bloque Nave(string _nombre, ...) dentro del código.
4. **Desafío Funcionalidad:** Explicá con tus palabras qué funcionalidad elegiste agregar de las 15 opciones y en qué partes del código tuviste que insertarla para que funcione.

7. ESCUDO ANTI-EXCUSAS (Resolución de dudas frecuentes)

Excusa típica	Respuesta y Acción Directa
"No me enseñaron a hacer un Manual de Usuario/Programador en 4to año."	Buscar. En las Actividades se adjuntó ejemplos en PDF/md (manualdelProgramadorv1.0.0.pdf, manualdelUsuariov1.0.0.pdf). Descargalos de Classroom/Moodle, leelos y usalos como plantilla cambiando los textos para que hablen de naves espaciales.
"El modelo APA v7 es muy confuso, no sé qué poner."	Revisar. Tienen el documento Plantilla APA LPR. Su informe debe tener exactamente 4 títulos: Introducción, Desarrollo (van las respuestas del cuestionario), Conclusión y Referencias. Ni más, ni menos.
"No tengo base de C++ y ahora nos piden Clases."	Falso. El código fuente se da tipeado y comentado. El objetivo es que <i>interpreten</i> la lógica técnica (pasaje por referencia, RAM, struct, etc.) y aprendan a documentarla como en la industria del software.
"Me copio/pego código de ChatGPT y listo."	Error. Si no está el repositorio GitHub/GitLab estructurado, el código documentado, el informe APA v7 con la plantilla exacta y los manuales redactados con criterio, la nota es Desaprobado.

PAUTA DE ENTREGA Y EVALUACIÓN

1. **Formato de Entrega sugerida del informe APA v7 en PDF:** Informe en PDF redactado bajo normas APA v7 guardado en docs/InformeEEST1_LPR2026_ACT08_G03_[ApellidoNombre]_Informe_v1.0.0.pdf. con **versionado** obligatorio.
2. **Manuales:** Guardados en docs/manuales/ con **versionado**.
3. **Demostración de Persistencia:** Deben incluir capturas de pantalla de la terminal jugando una partida, cerrando la consola, volviéndola a abrir y mostrando el historial persistido de la Opción 2.
4. **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub/GitLab con toda la estructura de carpetas (src/, docs/manuales, capturas/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas y/o archivos sueltos.
5. **Sincronización Git Dual-Remote:** Subir el código fuente y las capturas ejecutando la secuencia Dual-Push a GitHub y GitLab con el versionado indicado en la [Guía](#).
6. **Plataforma Moodle:** Adjuntar la URL pública de los repositorios [GitHub](#) y [GitLab](#) en el informe [APA V7](#) en PDF antes del plazo fijado.

Enlaces Técnicos de Referencia (Links de Consulta):

-  **cplusplus.com** — Clases y Objetos en C++: <https://cplusplus.com/doc/tutorial/classes/>
-  **GeeksforGeeks** — Modificadores de Acceso en C++:
<https://www.geeksforgeeks.org/access-modifiers-in-c/>
-  **Microsoft Learn** — Constructores e Inicialización de Clases:
<https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp>
-  **Git Documentation** — Flujo básico y manejo de repositorios: <https://git-scm.com/doc>

 **Fecha Límite de Entrega: 21 de Octubre de 2026.**

 Frase motivadora:

Un Técnico en Programación no es solo alguien que tipea hasta que el programa compila. Si el código funciona, se documenta. Hacer manuales y otra documentación no es un capricho, es lo que exige el mercado. Por eso lo entrenamos fuerte ahora en 5to. El profesionalismo se entrena hoy."